

JOSÉ ANTONIO FUENTES SANTIAGO

DOMAIN-DRIVEN DESIGN

EN UN

APOCALIPSIS ZOMBI!



16 bits graphics



Multiplayer

Software Development Book

Domain-Driven Design en un Apocalipsis Zombi (Demo)

José Antonio Fuentes Santiago

Bloque 1. Estrategia de supervivencia (diseño estratégico)



Imagen 1: El pueblo fue tomado por la horda...

Ahora...

Una explosión en medio de la noche te despierta. Escuchas gritos en la calle y te asomas por la ventana. Observas cómo poco a poco el caos se adueña de tu pueblo.

Pasan las horas lentamente mientras decides qué hacer. Alguien golpea apresuradamente tu puerta. Observas por la mirilla y ves que es tu amigo **Santi**. Rápidamente le abres la puerta. Nada más pasar cae al suelo fatigado. Tiene la cara pálida, la ropa desgarrada y un ligero corte en el brazo. Una vez que recobra el aliento te comenta lo que ocurre en el exterior.

Te dice que debéis abandonar el pueblo y buscar un refugio seguro. Al parecer los

militares han preparado uno a pocos kilómetros. Organizáis dos mochilas con todo lo que podéis llevar, principalmente alimentos y medicinas.

Salís a la calle pegados a la pared, con todos los sentidos en alerta. El olor se hace presente antes de que posáis ver el horror. El caos es absoluto, hay cuerpos ensangrentados por todas partes, unos inmóviles y otros alimentándose de los primeros. Va a ser complicado salir de ahí sin que os vean.

Pensáis en subir a tu coche pero hay una docena de vehículos cortando un lado de la calle y un camión volcado en la otra. Así que no os queda otra que caminar en silencio esquivando a esas cosas. Por suerte mientras se alimentan parecen ajenas a su entorno.

Tras un día de camino, muchas heridas y huidas precipitadas, conseguís llegar al refugio. **Nora** es la comandante que lo lidera. Bajo su mando trabajan varios equipos: sanidad, logística y comando.

Os llevan a una sala en la que os hacen un análisis completo, verificando que no tenéis ningún mordisco. Tras esto, os dan la bienvenida al que será vuestro nuevo hogar... de momento.

En este bloque...

Verás cómo integrarte en el equipo de supervivientes del refugio y aprender a hablar con sus mismos protocolos. Además conocerás las funciones de cada departamento y como estas quedan bien delimitadas.

- **Capítulo 1. Lenguaje ubicuo:** Aprende a usar las mismas palabras en el código y las conversaciones del equipo para eliminar malentendidos.
- **Capítulo 2. Contextos delimitados:** Descubre cómo dividir un sistema en fronteras donde cada modelo tiene un significado claro y coherente.
- **Capítulo 3. Mapa de contextos:** Analiza cómo se comunican los Contextos, quién manda sobre quién y qué tipos de relaciones existen.
- **Resumen del bloque 1:** Consulta las definiciones clave de Lenguaje Ubicuo, Contextos Delimitados y Mapa de Contextos.

Capítulo 1. Lenguaje ubicuo

Tras una breve entrevista, Nora os explica que perdieron a los integrantes del equipo informático del refugio por un fallo en los protocolos de seguridad. En ese momento, suena un golpe seco en la puerta de la sala. Entra **Joel**, el jefe de Sanidad, con cara de malas noticias.

—Nora, tenemos a alguien con fiebre en el módulo oeste. He pedido que lo lleven a aislamiento, pero en el equipo de Comando dicen que no tienen órdenes tuyas.

Nora exhala despacio.

—Joel, ya hablamos de eso otras veces. Si tiene cualquier síntoma compatible con ser un infectado va directo a cuarentena, sin esperar mi confirmación. Eso es una regla fija.

—Lo sé, pero ellos dicen que «cuarentena» y «aislamiento preventivo» no son lo mismo.

—Se equivocan, es exactamente lo mismo. —Se gira hacia vosotros antes de salir y tras un suspiro os comenta—. Si queremos sobrevivir aquí, vamos a tener que evitar estos malentendidos. —Señala los terminales del sistema que gestiona el centro—. ¡Seguidme! Os explicaré cómo funciona todo esto, y pasaremos a ver de qué sois capaces.

Lo que acaba de pasar entre Joel y el equipo de Comando —dos palabras distintas para la misma regla— ocurre también en el código que describe el refugio. En DDD, todo lo que el software intenta modelar —quién está infectado, qué suministros quedan, qué rutas son seguras— se llama **Dominio**. Y cuando el código deja de hablar en los mismos términos que el Dominio, se convierte en un acertijo.

El lenguaje ubicuo

El primer pilar de DDD es establecer un *lenguaje común* entre toda la gente que participa en un proyecto. En DDD eso se llama **Lenguaje Ubicuo** (*Ubiquitous Language*): las mismas palabras que usa Nora para describir cómo funciona el refugio aparecen también en el código y en las conversaciones del equipo. Así se ahorra el esfuerzo de realizar una traducción mental continua entre lo que dice el negocio y lo que has codificado.

Cuando se usa el Lenguaje Ubicuo, se deja de usar nombres como `Usuario` o `Persona` para utilizar algo más adecuado, por ejemplo `Superviviente`. Tampoco se define `setSalud()`, sino que se crean métodos como `sufrirHerida()` o `aplicarVendaje()`. Gracias a eso, no hace falta consultar la documentación para entender qué hace cada cosa.



Imagen 2: Nora y Joel se ponen de acuerdo para comenzar a hablar en un lenguaje común.

Cuando el código no habla el mismo idioma

Una vez que Nora termina de explicaros el funcionamiento del refugio, Santi se sienta ante el terminal de la sala de coordinación y empieza a revisar el sistema que el equipo anterior dejó montado. El refugio lleva días funcionando, pero hasta ahora nadie había tenido tiempo de poner orden en el código: los primeros instantes del apocalipsis estuvieron llenos de improvisación. A los pocos minutos Santi te llama.

—Oye, mira esto —dice Santi señalando la pantalla—. `ruta.nivel = 0`. ¿Qué significará?

—No tengo ni idea —respondes acercándote—. ¿La más segura? ¿La más corta? ¿La única disponible?

—Y aquí, `superviviente.estado = 2`. ¿Infectado? ¿En cuarentena? ¿Ambas cosas?

—Parece que no hay forma de saberlo sin rastrear todo el código —le respondes—. Y parece que quien lo escribió ya no está aquí para contarlo.

La consecuencia directa de no usar Lenguaje Ubicuo es la **pérdida de conocimiento del Dominio**. El código acaba convirtiéndose en una colección de números mágicos y nombres raros que sólo entiende quien los escribió. Y con suerte, porque a veces ni esa persona se acuerda dos meses después o pasó a mejor vida tras un ataque zombi.

Eso es exactamente lo que ocurrió aquí. Cada vez que Nora hablaba de “reabastecimiento

de urgencia”, el equipo anterior programaba `suministro.prioridad = 1`. Cuando decía “ruta segura”, en el código aparecía como `ruta.nivel = 0`. El equipo que había nunca escribió ese diccionario mental y ahora ha desaparecido con ellos.

Del lenguaje al código

El conocimiento del Dominio tiene que quedar escrito. No hace falta un documento de cien páginas, es suficiente con un glosario sencillo con los términos clave.

Para crear el glosario, graba reuniones con el equipo de negocio y anota cada término técnico que usen. Luego, valida con ellos que tu interpretación es correcta. Esto evita malentendidos desde el principio. En el capítulo 14 veremos este proceso de forma más detallada.



Consejo de superviviente

El glosario no es un documento que se escribe una vez y se archiva. El Dominio evoluciona: lo que hoy se llama “cuarentena” puede llamarse mañana “aislamiento preventivo”. Cuando un término cambia, el glosario y el código tienen que moverse juntos —si no, vuelves al problema inicial, con dos palabras para la misma cosa—. Trata los cambios de lenguaje con la misma disciplina que los cambios de código.

Santi lleva unos minutos averiguando el origen del código que tiene en pantalla:

```
ruta.nivel = 0;
```

Tras investigar y hacer pruebas, refactoriza el código de forma que comienza a utilizar Lenguaje Ubicuo.

```
ruta.marcarComoSegura();
```

Con `marcarComoSegura()` no hace falta preguntar qué sucede con la ruta. Es más, si marcar una ruta como segura implicara notificar al equipo de patrulla o actualizar el mapa, pondrías esa lógica dentro del método, evitando hacer cambios en quién lo llama.



Pista

Si el equipo de negocio usa términos que suenan raros o técnicos, no los traduzcas automáticamente. Pregúntales qué significan exactamente. Una misma palabra

como “suministro” puede significar cosas muy distintas según el contexto. Como diría Íñigo Montoya: «No creo que signifique lo que tú crees».

Capítulo 2. Contextos Delimitados

Tras unos días en el refugio, Nora os asigna a Santi y a ti la tarea de observar cómo trabaja cada departamento del refugio. Empezáis por la sala de coordinación, donde **Octavia** dirige las operaciones de Comando.

A los pocos minutos escucháis una discusión en el pasillo. **Andreu**, jefe de logística, se dirige a **Joel** con cara de pocos amigos.

—Necesito saber si Pedro puede hacer el trayecto hasta el punto de abastecimiento del norte —dice, dejando una carpeta sobre la mesa—. Llevo toda la mañana esperando el informe de sanidad.

—Pedro sigue en cuarentena —responde Joel—. A mí no me preocupa si puede caminar tres horas. Me preocupa saber si la fiebre que tiene es por un posible contagio de virus zombi. Si es así, podría poner en peligro a todo el refugio.

Andreu sólo quiere saber si Pedro puede llegar al norte sin desplomarse. Y por otro lado Joel quiere saber si dentro de dos días tendremos medio refugio infectado. Los dos hablan de Pedro, pero están intentando resolver problemas distintos. Podría parecer qué tener una sola ficha para Pedro sería lo más limpio. Pero los datos que necesita Andreu y los que necesita Joel no se parecen en nada.

¿Qué es un Contexto Delimitado?

En sistemas grandes, la misma palabra puede significar cosas completamente diferentes dependiendo de quién la use. Es ahí cuando entra en juego el **Contexto Delimitado**: una parte del proyecto con fronteras claras donde cada término tiene un único significado. Como cada departamento del refugio, lo que se habla dentro tiene sus propias reglas, y fuera de estos, esas mismas palabras pueden significar otra cosa.

En el refugio, la palabra “**Superviviente**” no significa lo mismo para todo el mundo.

- Para **Logística**, es alguien que consume recursos, tiene una ubicación y puede transportar suministros. Les interesa su resistencia y su capacidad de carga.
- Para **Sanidad**, es alguien con un historial médico, un nivel de salud y un riesgo de infección. Les importa su temperatura corporal y si se ha expuesto a mordeduras.
- Para **Comando**, es una unidad con habilidades específicas que puede asignarse a refugios o misiones. Les interesa su experiencia en combate o sus conocimientos técnicos.

Si intentas meter todo esto en un único modelo de `Superviviente`, acabarás con una clase gigantesca con decenas de propiedades, donde la mitad no tendrá sentido en cada Contexto. Y peor aún, cuando Logística quiera cambiar cómo se calcula la capacidad de carga, podría romper accidentalmente algo en Sanidad.

Con todo esto en mente, te sientas frente al terminal de la sala de coordinación y escribes los dos enfoques para que quede claro:

```
// "Superviviente Monstruo" que lo sabe todo y que hay que evitar.
```

```
interface Superviviente {  
    id: string;  
    nombre: string;  
    salud: number;           // Sanidad  
    vacunas: string[];      // Sanidad  
    pesoMaximo: number;     // Logística  
    ubicacion: Coordenada; // Logística  
    rango: string;          // Comando  
}
```

```
// Es mejor disponer de modelos específicos por contexto.
```

```
// En el contexto de sanidad:
```

```
interface SupervivienteSanidad {  
    id: string;  
    nivelInfeccion: number;  
    historial: NotaMedica[];  
}
```

```
// ...Y en el de logística:
```

```
interface SupervivienteLogistica {  
    id: string;  
    capacidadCarga: number;  
    estaDisponible: boolean;  
}
```

Un modelo por contexto

Si recuerdas el capítulo anterior, cada contexto, también tendrá su propio Lenguaje Ubicuo: cada departamento no solo modela a Pedro de forma distinta, sino que usa términos, acciones y conceptos que no existen en los demás. El lenguaje de Andreu y el de Joel no compiten: simplemente pertenecen a mundos distintos.

La solución es dividir el sistema en Contextos Delimitados, cada uno con su propio

modelo. Andreu habla de resistencia y capacidad de carga. Joel habla de historial médico, síntomas y riesgo de contagio. Octavia habla de habilidades, turnos y asignaciones. Cada departamento resuelve un problema distinto, y su **Superviviente** solo tiene las propiedades que le importan a ese problema.



Imagen 3: Los tres contextos delimitados de nuestro apocalipsis: comando, sanidad y logística.

Pista

¿Dónde acaba un Contexto y empieza el siguiente? Pregúntate: “¿este concepto cambia de significado en diferentes partes del negocio?”. Si “Superviviente” significa cosas distintas para Logística y para Sanidad, ahí tienes tu frontera: probablemente necesitas Contextos separados.

Andreu puede cambiar cómo calcula las rutas sin tocar el historial médico que usa Sanidad. Por otro lado, Joel puede añadir un nuevo campo al historial médico sin que Logística se entere. Cada equipo trabaja en su problema sin necesidad de coordinar cada cambio con el resto.

Consejo de superviviente

Sé que duplicar **SupervivienteSanidad** y **SupervivienteLogistica** te va a es-cocer: a todos nos enseñaron que duplicar es pecado. Pero compartir un modelo

de Dominio entre Contextos a la larga puede salir más caro, ya que cada cambio en un equipo puede afectar al otro. Duplica con cabeza y sin remordimientos.

Subdominios

Mientras Octavia os explica como se organiza Comando, observas el mapa del refugio fijado en la pared: rutas de patrulla, zonas de riesgo, puntos de abastecimiento. Hay muchos problemas que resolver aquí. Pero no todos tienen el mismo peso.

—¿Qué es lo más importante durante un ataque zombi? —pregunta Octavia, sin apartar la vista del mapa.

La pregunta te ha pillado de improviso. Parpadeas como volviendo a la realidad y después de un momento de duda respondes:

—¿La defensa?

—Exacto. En este caso, no nos sirve copiar lo que hace otro refugio. Ya que cada refugio tiene una serie de características específicas, ya sea por qué esté en una zona elevada, tenga agua alrededor, etcétera. Aquí tenemos este edificio militar con una serie de accesos protegidos. Nuestra estrategia de defensa no la puede diseñar nadie más. —Hace una pausa—. Lo siguiente a priorizar después de la defensa son las exploraciones. Pero cualquier equipo con experiencia puede seguir más o menos los mismos protocolos de reconocimiento. Y finalmente, las guardias nocturnas... —señala un papel con una tabla escrita a mano clavado en la pared— antes de enfermar, las gestionaba Pedro con una lista y un bolígrafo. No hace falta mucho más.

DDD llama a esas categorías **Subdominios**, y describen cómo se divide el mundo real antes de escribir una sola línea de código.

- **Dominio Principal** (*Core Domain*): lo que hace único a este refugio y no se puede copiar de otro. La estrategia de **Defensa** depende del terreno, los recursos y el entorno concreto: no es lo mismo defender un edificio en lo alto de una montaña que uno en un valle. Nadie más puede darte esa solución hecha. Aquí el código debe reflejar con mayor fidelidad las reglas del negocio, y aquí merece la pena aplicar DDD a fondo.
- **Subdominio de Apoyo** (*Supporting Subdomain*): necesario, pero no diferenciador. La **Exploración** es vital para la supervivencia, pero sigue protocolos de

reconocimiento que cualquier grupo organizado podría aplicar. Merece construirse bien, sin la misma obsesión que el Dominio Principal.

- **Subdominio Genérico** (*Generic Subdomain*): problemas estándar que cualquiera resuelve igual. La gestión de **Guardias** —turnos, horarios, relevos— no es estratégica ni única. Aquí tiene sentido usar soluciones ya hechas: librerías, servicios externos, o incluso una hoja de papel en la pared.

Consejo de superviviente

Invierte el mayor esfuerzo de diseño en el Dominio Principal. Aplicar DDD completo a un Subdominio Genérico es como construir una puerta blindada para el baño: es posible, pero implica aplicar un refuerzo extra allá donde no es necesario. Para una visión más compacta de Subdominios y Contextos Delimitados, ver bibliografía [3].



Imagen 4: Los tres tipos de Subdominio: Dominio Principal (Defensa), Subdominio de Apoyo (Exploración) y Subdominio Genérico (Guardias), con el tamaño del área representando el esfuerzo de diseño recomendado

Subdominios y contextos delimitados no son lo mismo

Acabas de ver tres Subdominios: Defensa, Exploración y Guardias. Pero antes, cuando hablamos de Contextos Delimitados, también apareció “Comando”. Podrían ser lo mismo con distinto nombre, pero no es así. Y entender por qué es clave para entender DDD.

Octavia señala el mapa de la sala de coordinación. Éste tiene rutas de exploración, zonas de riesgo y círculos de color rojo señalando los sectores ocupados por hordas.

—Esto es el refugio real. El problema que tenemos que resolver. —Luego se gira hacia los tres ordenadores apilados en la esquina—. Aquello es lo que hemos montado para gestionarlo.



Imagen 5: Comparación entre el espacio del problema (Subdominios Defensa, Exploración y Guardias en el mapa) y el espacio de la solución (un único Contexto de Comando que los agrupa)

La distinción que hace Octavia te ayuda a entenderlo. Los Subdominios son el mapa: describen qué problemas existen en el mundo real y cuál es su importancia estratégica. Defensa, Exploración y Guardias son tres problemas reales que hay que resolver antes de escribir una línea de código. Los Contextos Delimitados son los ordenadores: las fronteras que defines en el sistema para que cada modelo tenga un significado claro y no se mezcle con los demás.

Son dos capas distintas, sin jerarquía entre ellas. Un Subdominio describe un problema, mientras que por otro lado, un Contexto describe cómo decides resolverlo en código. La relación entre ambos no sigue ningún patrón fijo. Varios Subdominios pequeños pueden agruparse en un único Contexto. Un Subdominio muy grande puede partirse en varios Contexto. Incluso puede haber una correspondencia exacta de uno a uno. Es una decisión de diseño que depende del equipo, los recursos y la complejidad del problema.

—¿Veis que Defensa, Exploración y Guardias los gestiona el equipo de Comando?
—continúa Octavia—. No tiene sentido montar tres equipos separados para problemas tan relacionados entre sí, con el personal que tenemos. Así que Comando termina englobando los tres.

Saber qué es Dominio Principal y qué no lo es te permite decidir dónde poner el foco, y dónde no malgastarlo.

Sin embargo, los Contextos no son islas. Cuando Joel detecta un brote, Octavia necesita saberlo para activar una evacuación. Cuando Andreu actualiza las existencias, Comando debe conocer si hay escasez crítica. ¿Cómo se comunican sin romper sus fronteras ni contaminar sus modelos? Porque si se equivocan, pueden condenar al refugio.

Continuará...

Hasta aquí la demo. Has llegado al refugio, has aprendido a hablar con las mismas palabras que el equipo que lo defiende y has visto por qué un «superviviente» no significa lo mismo para Joel que para Andreu. Con eso ya sabes trazar fronteras entre modelos. Lo que todavía no sabes es cómo cruzarlas sin romperlas: quién manda sobre quién cuando Sanidad detecta un brote y Comando necesita saberlo, y qué tipos de relación existen entre Contextos. Eso es lo que te queda por aprender antes de cerrar este primer bloque: el mapa de contextos.

El libro completo sigue mucho más allá. En el segundo bloque el grupo toma un centro comercial y lo convierte en refugio, y por el camino aprendes a distinguir Entidades de Objetos de Valor, a proteger reglas con Agregados y a colocar la lógica que no encaja en ninguno de ellos. En el tercero hay que coordinar dos refugios a la vez, y ahí entran los Repositorios, la capa de aplicación, los Eventos de Dominio y la Arquitectura Hexagonal. El cuarto bloque es el más duro: el centro comercial cae, toca evacuar a una cárcel y resistir la horda final. Para eso hacen falta Sagas, saber reconocer los antipatronos que hunden un proyecto, probar el Dominio sin miedo y descubrir el modelo con el equipo antes de escribir código. Al final hay tres apéndices de consulta: cuándo usar DDD y cuándo no, cómo se aplica en otros dominios y las preguntas que más se repiten.

Si te ha gustado lo que has leído hasta ahora, puedes conseguir el libro completo:

- [Tapa blanda](#)
- [eBook](#)

Revisa las tiendas de Amazon de otros países para encontrarlo también.